

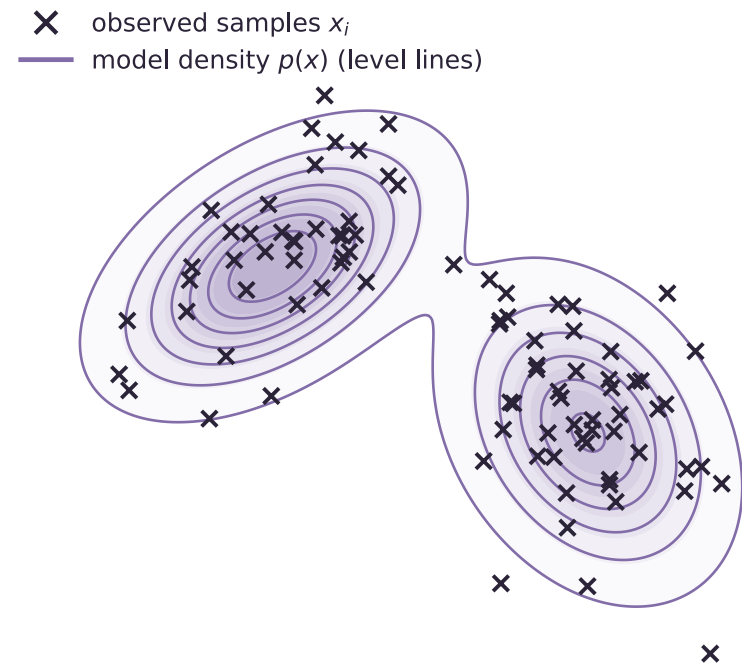
Deep Learning

6. Generative neural networks

Romain Tavenard

Generative models in a nutshell

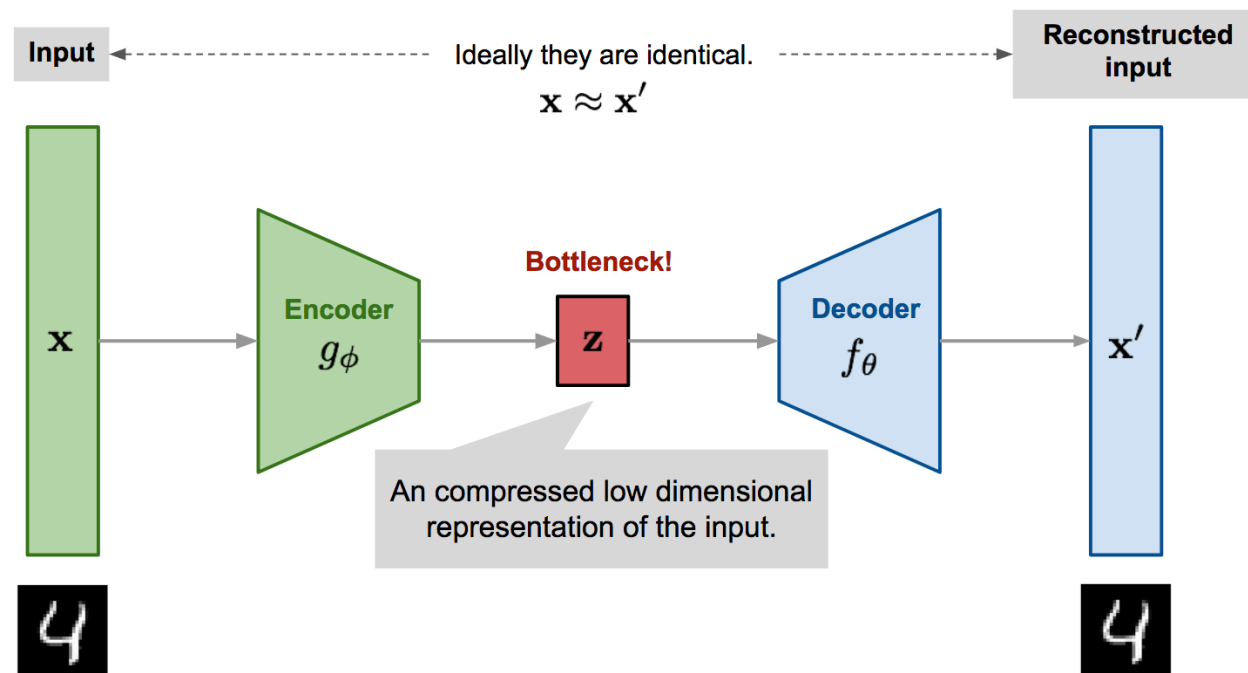
- Goal: model $p(x)$ or $p(x|y)$
 - explicitly
 - eg. Gaussian Mixture Models
 - **implicitly**
 - at least allow for sampling (*i.e.* generate new data)
 - e.g. Variational Auto Encoders, Generative Adversarial Networks



GMM with 2 components

Auto-encoders [Hinton & Salakhutdinov, 2006]

- Encode information in a latent space
 - typically lower-dimensional
 - similar in spirit to a non-linear PCA
 - not a generative model *per se*!



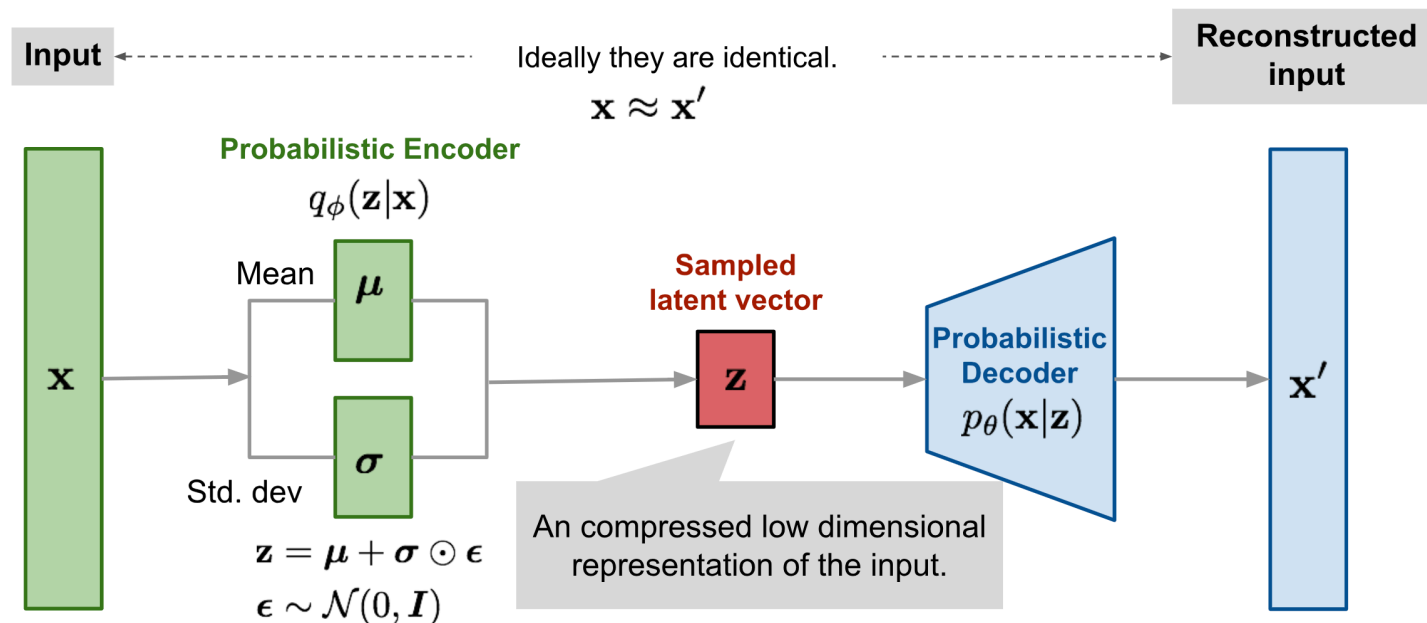
Source: lilianweng.github.io

Variational auto-encoders [Kingma & Welling, 2014]

- **Goal:** turn auto-encoders into generative models
- **Idea:** prior $p(\mathbf{z}) = \mathcal{N}(0, \mathbf{I})$ on latent space, enforced by KL:

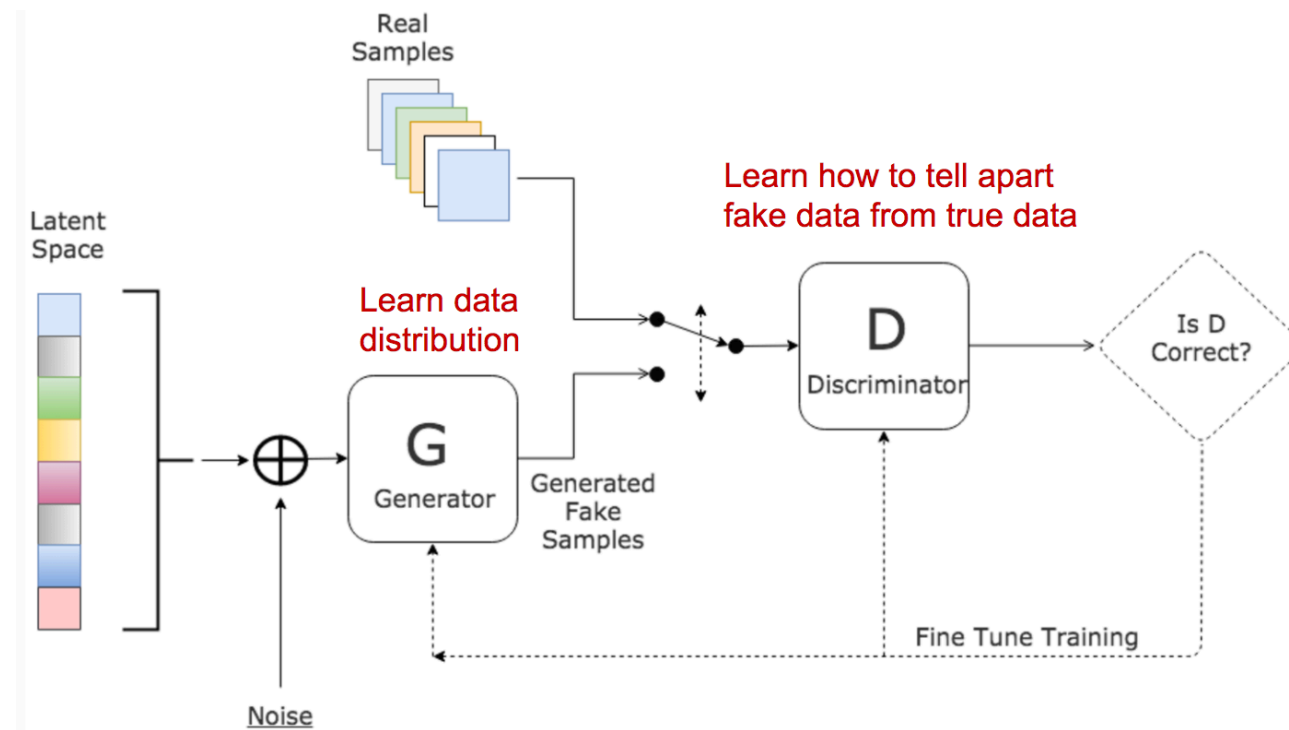
$$\mathcal{L} = \underbrace{-\log p_{\theta}(\mathbf{x}|\mathbf{z})}_{\text{reconstruction}} + \underbrace{\text{KL}(q_{\Phi}(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))}_{\text{stay close to prior}}$$

- **Generative process:** draw $\mathbf{z}$ from prior $\rightarrow$ decode



Source: lilianweng.github.io

GANs [Goodfellow et al., 2014]



Source: KDNuggets.com

- Two networks compete
 - D (discriminator): outputs $P(\text{sample is real})$
 - G (generator): turns noise $\rightarrow$ fakes
 - D wants to be right, G wants to fool $D \rightarrow$ **min-max**

GANs [Goodfellow et al., 2014]

- Loss function:

$$\min_G \max_D L(D, G) = \underbrace{\mathbb{E}_{x \sim p_r(x)} [\log D(x)]}_{\text{D right on real data}} + \underbrace{\mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]}_{\text{D right on fakes / G wants this small}}$$

- ▶ $\max_D$: pushes both terms up (sharper discriminator)
- ▶ $\min_G$: pushes 2nd term down ($D(G(z)) \rightarrow 1$, fakes look real)
- Optimization
 - ▶ Alternate between training G and D
 - ▶ Very unstable in practice
- Generative process
 - ▶ Draw z from p_z (random noise)
 - ▶ Pass it to the generator to compute $G(z)$ (fake sample)

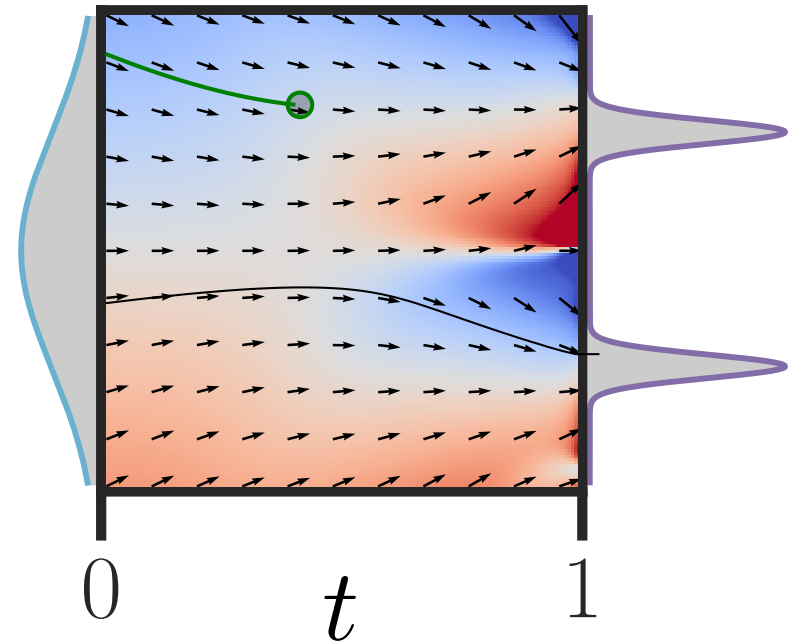
Conditional Flow Matching [Lipman et al., 2023]

- **Principle:** learn a vector field transporting noise ($t=0$) $\rightarrow$ data ($t=1$) by minimizing

$$\mathbb{E}_{x_0, x_1, t} \| u^\theta(x_t, t) - (x_1 - x_0) \|^2$$

- At generation time, use an Ordinary Differential Equation solver (eg. Euler scheme):

$$x_{t+\varepsilon} \leftarrow x_t + \varepsilon u^\theta(x_t, t)$$



Learned velocity field transports noise ($t=0$) to data ($t=1$).

Source: Emonet et al. “A Visual Dive into Conditional Flow Matching”, ICLR 2025